A Set-based Semantics for Validating ASN.1 Specifications

Christian Rinderknecht

Network Architecture Laboratory
Information and Communications University
58-4 Hwaam-dong, Yuseong-gu, Daejeon
305-732, South Korea
rinderkn@icu.ac.kr

March 28, 2002

Network Architecture Laboratory Seminar

Plan

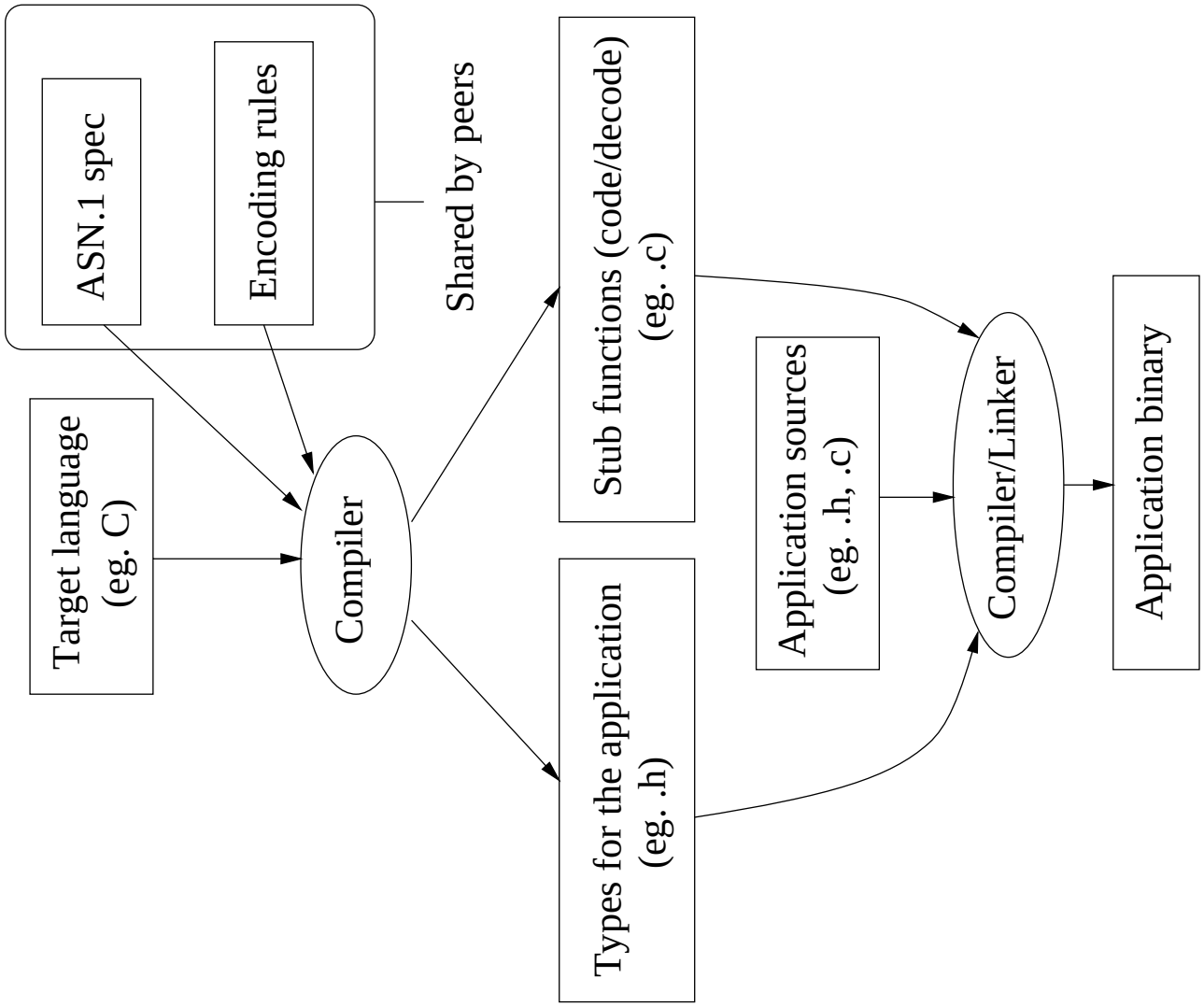
- Introduction
- Short Presentation of ASN.1
- Issues in Validation
- A Constraint-based Solution
- Conclusion

Introduction

Abstract Syntax Notation One (ASN.1) is a standard language for defining data types whose values may be exchanged across a network between two communicating applications, independently from the possible heterogeneity of the peers.

It is used in network management, secure email, mobile telephony, air traffic control, voice and video over the Internet, electronic commerce, digital certificates, secure email, radio paging, interactive television, financial service systems, biometrics, ATM transactions, 800-number call routing to local carriers (USA), plane take-offs and landings, Federal Express tracking network, Microsoft Internet Explorer, NetMeeting, Outlook, wireless applications from Nokia, Ericsson and Motorola, databases (genetics, libraries), diagnostic monitoring systems (car factories), subway equipments, etc.

The Encoding Rules specify how to serialize an ASN.1 value.


ASN.1 spec
Encoding rules
Shared by peers
Target language
(eg. C)
Compiler
Stub functions (code/decode)
(eg. .c)
Types for the application
(eg. .h)
Application sources
(eg. .h, .c)
Compiler/Linker
Application binary

Short Presentation of ASN.1

Basic types

- `ok BOOLEAN ::= TRUE`
- ```
  zero INTEGER ::= 0
  DayInTheYear ::= INTEGER {first(1), last(356)}
  newYearsEve DayInTheYear ::= last
  ```
- ```
  SynchroIndicator ::= ENUMERATED {serial, parallel}
  synchro SynchroIndicator ::= serial
  ```
- ```
  pi REAL ::= 3.14159
  micron REAL ::= {mantissa 1, base 10, exponent -6}
  ```
- ```
  byte BIT STRING ::= 'OD'H  -- or '00001110'B
  T ::= BIT STRING {msb(7), lsb(0)}
  v T ::= {msb, lsb}  -- or '10000001'B
  ```
- For historical reasons, there are many string types.

Constructed types

– The `SET` type corresponds to the record-like structures in programming languages:

```
PersonInfo ::= SET {age INTEGER,married BOOLEAN}
i PersonInfo ::= {married FALSE, age 32}
```

but also some fields can be marked as optional or having a default value:

```
Point ::= SET {x REAL DEFAULT 0, y REAL DEFAULT 0}
origin Point ::= {}              -- or {x 0.0, y 0.0}
```

A real example:

```
DataAcknowledgementTPDU ::= SET {
  destRef         Reference,
  yr-tu-nr        TPDUnumber,
  checkSum        CheckSum OPTIONAL,
  subSeqNr        SubSequenceNumber DEFAULT 0,
  flowControlCnf FlowControlConfirmation OPTIONAL}
```

– The `SET OF` type corresponds to the mathematical notion of sets with repetition:

```
T ::= SET OF INTEGER
empty T ::= {}
small T ::= {7, 9, 1, 1, 3}
```

– The `CHOICE` type corresponds to a **union** in C or a **case** in Pascal.

```
T ::= CHOICE {x REAL, y BOOLEAN}
u T ::= x : 0.5
v T ::= y : FALSE
```

The Protocol Data Units (PDU) are `CHOICE` types, because they model all the possible queries and responses between two peers. A `CHOICE` type may be recursive, like the other constructed types. One real example (a Network Management Protocol) is:

```
CMISFilter ::= CHOICE {
  item  FilterItem,
  and   SET OF CMISFilter,
  or    SET OF CMISFilter,
  not   CMISFilter}
```

Subtyping constraints

ASN.1 offers a very involved subtyping paradigm consisting of constraints upon recursive types, that restricts their corresponding sets of values in a set-theoretic manner, but also in a structural way.

– Interval Constraint: the `INTEGER`, `REAL` and (almost all) string types have totally ordered values, hence allowing interval definitions. For instance:

```
PositiveOrZeroInteger ::= INTEGER (0..MAX)
PositiveInteger ::= INTEGER (0<..MAX)
NegativeOrZeroInteger ::= INTEGER (MIN..0)
NegativeInteger ::= INTEGER (MIN..<0)
PositiveReal ::= REAL (0<..PLUS-INFINITY)
NegativeReal ::= REAL (MINUS-INFINITY..<0)
RealInterval ::= REAL (4e-5..1e-4)
```

– Value Constraint: to restrict the set of values of a type to be a singleton:

```
Wednesday ::= Day (wednesday)
```

– Union Constraint: the new subtype contains the values of the first subtype and of the second subtype (keyword `UNION` or symbol `|`):

```
Day ::= ENUMERATED {monday, tuesday, wednesday,
                    thursday, friday, saturday, sunday}
WeekEnd ::= Day (saturday | sunday)
```

– Alphabet Constraint: the strings can be restricted to be built upon a given alphabet:

```
CapitalAndSmall ::=
  IA5String (FROM ("A".."Z" | "a".."z"))
CapitalOrSmall ::=
  IA5STring (FROM ("A".."Z") | FROM ("a".."z"))
```

– Size Constraint: the values of string types may be constrained to a given sizes, introducing a constraint by the keyword `SIZE`:

```
Exactly31BitsString ::= BIT STRING (SIZE (31))
StringOf31BitsAtTheMost ::= BIT STRING (SIZE (0..31))
NonEmptyString ::= BIT STRING (SIZE (1..MAX))
```

The size constraint can also apply to `SET OF` types. In that case, the semantics is very different: the values of the types are sets whose *cardinals* are specified by the size constraint:

```
SetOf5Strings ::= SET (SIZE(5)) OF PrintableString
SetOfStringsOf5Char::= SET OF PrintableString (SIZE(5))
```

– Intersection Constraint: the new subtype contains the values that belong to the two subtypes (keyword `INTERSECTION` or symbol `^`):

```
FrenchPhoneNumber ::=
  NumericString (FROM ("0".."9") ^ SIZE (10))
```

– Inclusion Constraint: to restrict a subtype to have only the values of a given subtype (optional keyword `INCLUDES`):

```
LongWeekEnd ::= Day ((INCLUDES (WeekEnd)) | monday)
Bis ::= Day (WeekEnd | monday)
```

– Complement Constraint: to restrict the values of a type to *not* belong to another subtype:

```
Lipogramme ::= IA5String (FROM (ALL EXCEPT ("e" | "E")))
```

– Constraint on `SET OF`: to restrict the elements of a `SET OF` value:

```
TextBlock ::= SET OF VisibleString
AddressBlock ::= TextBlock (WITH COMPONENT (SIZE(1..32)))
```

– Partial Constraint: to restrict *some* fields of a `SET` or `CHOICE`:

```
Quadruple ::= SET {
  alpha ENUMERATED {in, out} OPTIONAL,
  beta  IA5String OPTIONAL,
  gamma SET OF INTEGER,
  delta BOOLEAN DEFAULT TRUE}
```

we can derive a subtype whose component `alpha` is always present and equals `in`, and the component `gamma` always has five elements:

```
Quadruple1 ::=
  Quadruple (WITH COMPONENTS {...,
                              alpha (in) PRESENT,
                              gamma (SIZE (5))})
```

This subtype has the same values as:

```
Quadruple1 ::= SET {
  alpha ENUMERATED {in, out} (in),
  beta  IA5String OPTIONAL,
  gamma SET SIZE (5) OF INTEGER,
  delta BOOLEAN DEFAULT TRUE}
```

Issues in Validation

ASN.1 types are considered as sets of values, thus types must have at least one (finite) value.

Because of the great expressivity of ASN.1, the compilers are not likely to fully check arbitrary combinations of subtyping constraints!

Vendors' argument: "This is hardly a real problem because those critical specifications are rarely found in practice."

My argument: "Why not solve completely the problem (for X.680) and get a better product?"

Preliminaries

First, we rewrite every X.680 specification into a subset of X.680 which has the same expressiveness as X.680, though having less ambiguities and syntactical constructs (formal definitions are easier too).

Especially, for each value declaration, we extract the given type and create a corresponding type declaration for it. We also create another type declaration where the previous type is *constrained* in order to contain the originally declared value. For instance:

```
y INTEGER (0..9) ::= 1  ⟶  { y A ::= 1
                           { A ::= INTEGER (0..9)
                           { B ::= A (y)
```

where `A` and `B` are fresh type references.

Problems

At this stage it can happen that

1. the types may have only infinite values, as `T ::= SET {a T}`;
2. some value declarations may be ill-typed, as `v REAL ::= ""`;
3. especially, some value references may be ill-typed, as
   `a VisibleString ::= b  b INTEGER ::= 0`;
4. the subtype constraints may be inconsistent: `T ::= REAL (SIZE(7))`;
5. the subtypes may be empty, as
   `T ::= SET ((SIZE (1)) ^ (SIZE (2))) OF REAL`;
6. the subtypes may have no value set: `T ::= REAL (ALL EXCEPT T)`.

In other words, the issues to settle are respectively:

1. the finiteness problem;
2. the typechecking problem;
3. the type compatibility problem;
4. the constraint consistence problem;
5. the non-emptyness problem;
6. the solvability problem.

Problem analysis

- type compatibility $\subseteq$ typechecking;
- constraint consistence, non-emptyness $\subseteq$ solvability (the system is solved when we construct explicitly the values of each subtype);
- typechecking $\subseteq$ solvability (we added initially a new type declaration for each value declaration).

So, finiteness and solvability are enough to get a full validation of X.680.

A Constraint-based Solution

Well-founded Types

A *well-founded type* is a type uniquely defined and which has at least one finite value.

We give an algorithm to check whether a type is well-founded. Basically, it analyses precisely the recursions.

Note: the well-foundedness of a type does *not* imply the well-foundedness of its subtypes:

```
T ::= CHOICE {a T, b REAL}
U ::= T (WITH COMPONENTS {..., b ABSENT})
```

type `T` is well-founded but `U` is not.

Set Constraints

Sets constraints are inclusions between expressions interpreted over the domain of sets of trees which may be recursively defined.

The ASN.1 types, by having a tree structure (technically, they are rational trees), fit perfectly in the usual domain of set constraints.

A set constraint has the form $\alpha \subseteq \beta$, where α and β are *set expressions*. Examples are $\mathsf{0}$ (the empty set), $\mathsf{1}$ (the set of all terms), α (a set-valued variable), $c(\alpha, \beta)$ (a constructor application), and the union, intersection or complement of set expressions. Given a set of constructors $\mathcal{C}$, where each $c \in \mathcal{C}$ has arity $a(c)$, set expressions are defined by the grammar:

$$\mathrm{E} ::= \mathsf{0} \mid \mathsf{1} \mid \alpha \mid c(\mathrm{E}_1, \ldots, \mathrm{E}_{a(c)}) \mid \mathrm{E}_1 \cup \mathrm{E}_2 \mid \mathrm{E}_1 \cap \mathrm{E}_2 \mid \neg \mathrm{E}_1$$

Common Application

Set constraints are used in program analysis: sets of terms describe the possibly computed values. Set constraints are generated from the program text and solving the former yields some useful information about the latter (e.g. type-ckecking or optimisation).

For example, consider a program with two uses of a variable v. Assume that from one use we determine that v must be a number (i.e. $v \subseteq \mathsf{Int} \cup \mathsf{Float}$) and from the other use that v cannot be an integer (i.e. $v \subseteq \neg\mathsf{Int}$). Combining these constraints, we can infer that v must be a floating point number (i.e. $v \subseteq (\mathsf{Int} \cup \mathsf{Float}) \cap \neg\mathsf{Int} = \mathsf{Float}$).

Application to ASN.1

In order to apply the set constraints to ASN.1 we only define specific constructors c that model all the subtyping constraints:

type $construct_1 =$ [‘Null | TRUE | FALSE]
and $construct_2 =$
  [‘Enum **of** *string* | ‘Regexp **of** *string* | *integer* | *real* | $construct_1$]
and *real_interval* =
  [_ _ .. _ _ **of** *real_bound* × *in_out* × *in_out* × *real_bound*]
and *real_bound* = [*real* | ‘MinInfReal | ‘PlusInfReal]
and *closed_int_interval* = [‘Interval **of** *int_bound* × *int_bound*]
and *int_bound* = [*integer* | ‘MinInfInt | ‘PlusInfInt]
and *list* = [‘Cons **of** E × *list* | ‘Nil]
and *map* = [‘Bind **of** *identifier* × E × *map* | ‘Nil]
and *construct* = [*list* | *map* | _ : _ **of** *label* × E
  | *real_interval* | *closed_int_interval* | $construct_2$]

As a special case, we define some useful set constants:

let $\mathbb{N} = \text{`Interval}\,(\text{`MinInfInt}, \text{`PlusInfInt})$
and $\mathbb{N}^+ = \text{`Interval}\,(\text{`PosInt}\,(0), \text{`PlusInfInt})$
and $\mathbb{R} = \text{`MinInfReal} < .. < \text{`PlusInfReal}$

Now the problem is to extract set constraints from each subtype, such as their solving provide a mapping from the subtype to its set of values: this is a *denotational semantics* of ASN.1.

As far as validation is concerned, if some constraints are unsolvable or the set of solutions is empty, them the specification must be rejected.

Constraint Collection

From Types

$$
\begin{aligned}
[\![\textsc{Integer}]\!]_\alpha &= (\alpha \doteq \mathbb{N}) \\
[\![\textsc{Real}]\!]_\alpha &= \alpha \doteq \mathbb{R} \\
[\![\textsc{Boolean}]\!]_\alpha &= \alpha \doteq \textsc{True} \dot{\cup} \textsc{False} \\
[\![\textsc{Enumerated}\,[a]]\!]_\alpha &= \alpha \doteq \text{`}\mathsf{Enum}\,(a) \\
[\![\textsc{Enumerated}\,(a :: b :: \mathrm{I})]\!]_\alpha &= \textbf{let } \beta \textit{ be a fresh variable} \\
&\quad \textbf{in } [\![\textsc{Enumerated}\,(b :: \mathrm{I})]\!]_\beta \\
&\quad\quad \wedge\ \alpha \doteq \text{`}\mathsf{Enum}\,(a) \dot{\cup} \beta
\end{aligned}
$$

et cetera.

From Subtypes

Similarly, constraints are collected from (proper) subtypes.

Constraint Solving

Aiken and Wimmers proposed the first solving algorithm for the class of constraints we are using:

References

[1] Alexander Aiken and Edward Wimmers. Solving Systems of Set Constraints. In *Seventh Annual IEEE Symposium on Logic in Computer Science*, pages 329–340, Santa Cruz, California, June 1992. Extended Abstract.

Conclusion

- First formal and complete semantics of ASN.1 (X.680). It is complementary to the usual syntax-driven informal approach.
- First algorithm for the full validation of ASN.1 (X.680).
- Full validation of ASN.1 means full interoperability at the data level between network equipments.
- The algorithm is given in a meta-language whose semantics already exists and there is a programming language for it (namely Objective Caml, cf. `http://www.ocaml.org` and `http://caml.inria.fr/`).
- Best site about ASN.1: `http://asn1.elibel.tm.fr/`